

---

W BOOTH SCHOOL OF ENGINEERING PRACTICE AND TECHNOLOGY

# SEPT Interactive Laboratory Platform

**Phase A — Developer Guide.** How to run the platform on your own machine from WebStorm, drive every part of it by hand, and satisfy yourself that it does what it claims — written for someone who has never started it before.

---

**REPOSITORY**

Majd-214/SEPT-ILP

**BRANCH**

claude/phase-a-platform

**PILOT COURSE**

SMRTTECH 3CC3

**DOCUMENT DATE**

17 August 2026

**AUDIENCE**

**STATUS**



# What is in this guide

Sections 1–4 get the platform running and publish a site. Sections 5–9 walk you through each feature as its real user would. Section 10 is the checklist that proves the whole thing works. Keep 11 and 12 for when something breaks.

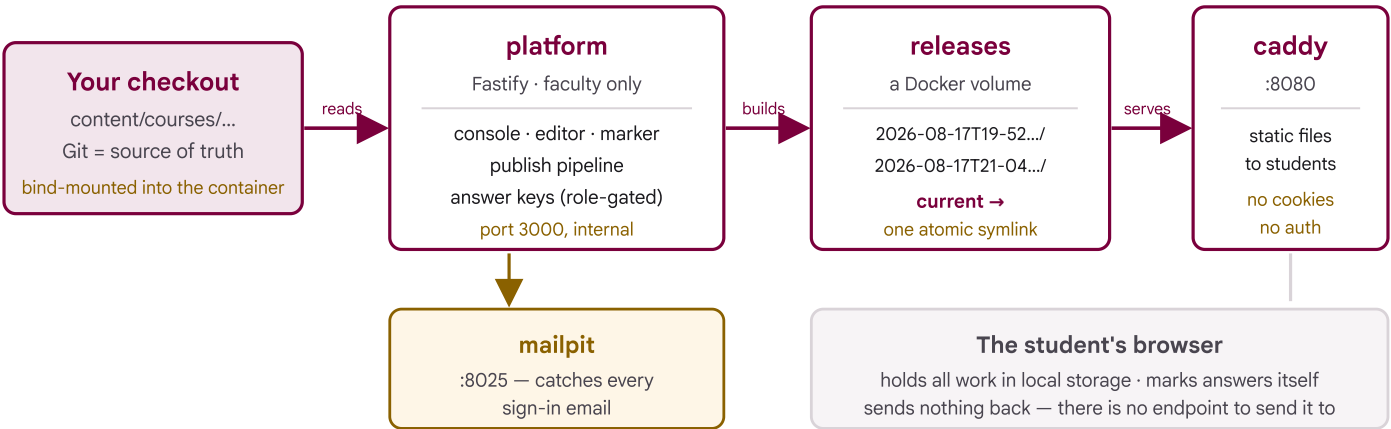
|    |                       |                                                      |
|----|-----------------------|------------------------------------------------------|
| 0  | The mental model      | Four moving parts, one paragraph                     |
| 1  | Before you start      | Docker, WebStorm, disk, ports                        |
| 2  | First run             | Starting the stack from WebStorm                     |
| 3  | Signing in            | Magic links and Mailpit — there are no passwords     |
| 4  | Publishing            | The quality gates, and making the site exist         |
| 5  | Being a student       | Gating, progress files, the submission package       |
| 6  | Editing content       | Draft → validate → commit → publish                  |
| 7  | Marking submissions   | The auto-marker and its exports                      |
| 8  | Single-file exports   | One HTML file, no server                             |
| 9  | Running the tests     | Four suites, 86 tests, one command                   |
| 10 | Proving it works      | The acceptance checklist, by hand                    |
| 11 | When something breaks | Symptom → cause → fix                                |
| 12 | Reference             | URLs, variables, repo map, invariants                |
| 13 | Honest status         | What is solid, what is stubbed, what you must decide |

## IF YOU ONLY READ ONE PAGE

Start Docker Desktop. In WebStorm open `deploy/docker-compose.yml` and click the double-arrow in the left gutter. Wait for the log to say *listening*. Open `localhost:8025`, click the sign-in link in the newest email, then press **Publish all courses**. That is the entire loop; everything after section 4 is detail.

# 0 The mental model

Four things are running, and they have a strict pecking order: your Git checkout is the truth, a build turns it into a frozen release, and a web server hands that release to students. Nothing flows backwards.



A gate failure stops the arrow at “builds”. The live symlink never moves, so students keep the last good release.

## Who touches what

| Person     | Where they go           | What they can do                                                                     |
|------------|-------------------------|--------------------------------------------------------------------------------------|
| Student    | :8080/c/<course>/<lab>/ | Everything, with no account. Work lives in their browser and in files they download. |
| Instructor | :8080/login             | Their own courses: edit content, read answer keys, download exports, run the marker. |
| Admin      | :8080/login             | All of the above, everywhere, plus inviting people and pressing Publish.             |

### THE ONE RULE WORTH INTERNALISING

The server has no route that accepts student data. Not a disabled one — none exists, and a test asserts it. When you are wondering “where did that student’s answer go?”, the answer is always: into their browser, and into the file they downloaded.

# 1 Before you start

Five minutes of checking now saves an hour of confusing errors later. Work down this list before you press anything.

| Requirement                            | How to check                                                                                                            | Why it matters                                                                                                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Docker Desktop is running              | Whale icon in the tray/menu bar is steady, not animating                                                                | WebStorm's Docker plugin drives Docker Desktop; it cannot start it for you.                                                                                         |
| WebStorm sees Docker                   | Settings → <i>Build, Execution, Deployment</i> → <i>Docker</i> ; the connection should say <i>Connection successful</i> | Without it, the run buttons in section 2 are missing entirely.                                                                                                      |
| The checkout is on a shared drive      | Docker Desktop → Settings → Resources → File Sharing includes the folder holding <code>SEPT-ILP</code>                  | The repository is mounted into the container. If it is not shared, the container starts and then fails to find any content.                                         |
| ≈ 6 GB free disk                       | Docker Desktop → Settings → Resources                                                                                   | The platform image is built on Microsoft's Playwright image (browsers included) — it is large, and it is what lets the accessibility gate run inside the container. |
| Ports 8080 and 8025 are free           | <code>lsof -i :8080</code><br>(macOS/Linux) · <code>netstat -ano   findstr :8080</code><br>(Windows)                    | Both are published to your machine. A conflict shows up as a container that exits immediately.                                                                      |
| Node 22 or newer <span>OPTIONAL</span> | <code>node --version</code>                                                                                             | Only needed for section 9 (running tests outside Docker). The platform itself never uses your Node.                                                                 |

## WINDOWS: LINE ENDINGS

Git on Windows rewrites text files to CRLF by default, which breaks any file Linux executes. The repository now carries a `.gitattributes` that pins LF, and the image strips stray CR bytes as a second line of defence — but if you cloned *before* that landed, normalise your working copy once:

```
git rm --cached -r .
git reset --hard
```

Symptom if you skip it: `exec /entrypoint.sh: no such file or directory` — which is Linux's confusing way of saying the shebang line ends in an invisible carriage return.

## Get the branch

```
# in WebStorm: Git → Branches → claude/phase-a-platform → Checkout  
git fetch origin  
git checkout claude/phase-a-platform  
git pull
```

## 2 First run

One file starts everything. The first run is slow and the rest are quick — that difference is normal and worth understanding, so you do not kill it half way through thinking it has hung.

- 1 Open `deploy/docker-compose.yml` in the editor.

WebStorm recognises it and puts a green **double-arrow** in the left gutter beside the top-level `services:` key, with single arrows beside each service.

- 2 Click the double-arrow, and choose *Run 'docker-compose.yml: Compose Deployment'*.

Equivalently, from a terminal: `cd deploy && docker compose up --build`. WebStorm's Services tool window (`Alt + 8`, or `⌘8` on a Mac) now shows all three containers with live logs.

- 3 Wait, and watch the `platform` container's log.

It proceeds through five phases, in this order:

| Phase                                  | First run | Later runs                        |
|----------------------------------------|-----------|-----------------------------------|
| Build the image (pull Playwright base) | 3–8 min   | skipped (cached)                  |
| Pull Mailpit and Caddy                 | < 1 min   | skipped                           |
| <code>npm ci</code> in three packages  | 1–3 min   | skipped unless a lockfile changed |
| Seed the accounts                      | seconds   | seconds (it is idempotent)        |
| Start listening                        | instant   | instant                           |

- 4 Look for this, and you are up:

```
installing dependencies in . ...
seeding (idempotent) ...
seed: created admin@demo (admin) and instructor@demo (instructor for
smrttech-3cc3, demo-101); sign-in links are in Mailpit.
starting platform ...
SEPT-ILP platform listening on http://0.0.0.0:3000
```

# Your three windows

| Open this            | And you get                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------|
| localhost:8080/login | The faculty console. Empty until you sign in — which needs the next section.                    |
| localhost:8025       | Mailpit. Every email the platform sends lands here instead of the internet.                     |
| localhost:8080       | The student site. Right now it says nothing is published — correct, you have not published yet. |

## STOPPING AND STARTING

**Stop:** the red square in Services, or `docker compose down` . Your accounts and releases survive in a Docker volume. **Start again:** same run button — seconds this time. **Full reset** (forget every account and release, keep all content): `docker compose down -v` .

### 3 Signing in

There are no passwords anywhere in this system, and that is deliberate. You ask for a link, the link arrives by email, and clicking it starts a session. When McMaster single sign-on arrives in Phase B it replaces exactly this one step and nothing else.

**1 Open Mailpit at `localhost:8025`.**

Two emails are waiting from the seed: one to `admin@demo`, one to `instructor@demo`.

**2 Open the one addressed to `admin@demo` and click the `/auth/...` link.**

You land on the console dashboard, signed in as an administrator.

**3 Need another link? Go to `localhost:8080/login`, type the address, submit.**

A fresh link appears in Mailpit. Links are single-use and expire after 30 minutes, so this is routine, not a failure.

### What the console holds

| Nav item   | Who sees it | Purpose                                                                                        |
|------------|-------------|------------------------------------------------------------------------------------------------|
| Dashboard  | everyone    | Course list, the Publish button, and the log of every publish run.                             |
| Invites    | admins      | Add a colleague and choose which courses they can touch.                                       |
| Link sheet | everyone    | The canonical URL for each lab, with copy buttons and a suggested Avenue to Learn topic title. |
| Exports    | everyone    | Download single-file labs and course ZIPs from the live release.                               |
| Editor     | everyone    | The content CMS (section 6).                                                                   |
| Marker     | everyone    | The auto-marker (section 7).                                                                   |

### Try the role boundary

Sign in as `instructor@demo` in a private window and confirm the console is narrower: *Invites* is gone, and the Publish button is replaced by a line explaining that publishing is an administrator's job. Both are enforced server-side, not merely hidden.

#### INVITING A REAL PERSON LATER

In production the address must be one they can actually receive mail at, and `MAIL_FROM` / `SMTP_HOST` must point at a real mail relay instead of Mailpit. Everything else about the flow is identical.

## 4 Publishing

Publishing is the only way content reaches students, and it is a one-way door guarded by six checks. If any check fails, students keep seeing exactly what they saw before.

1 On the dashboard (as admin), press **Publish all courses**.

2 The run appears in the table below, marked *running*. Open its report.

Give it two to six minutes: it builds both demo courses and runs a real browser over every page for the accessibility scan. Refresh the report to follow along.

3 A finished run shows **OK** and a gate report like this:

```
Building /app/content/courses/smrtech-3cc3 ...
gate inline-style      ... pass
gate class-allowlist   ... pass
gate answer-leak       ... pass
gate accessibility     ... pass
gate link-integrity    ... pass

Build complete: /data/releases/2026-08-17T21-04-11-903Z/smrtech-3cc3/site
```

### What each gate refuses to let through

| Gate            | Blocks                                                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| schema          | Content that does not match the JSON schemas — a quiz with no correct option, an image with no alternative text, a checkpoint with no way to complete it. |
| inline-style    | Any <code>style=</code> attribute. All styling must come through the design system, so the whole platform can be restyled from one place.                 |
| class-allowlist | CSS classes the stylesheet does not define — the usual cause of a component that silently renders unstyled.                                               |
| answer-leak     | Any expected answer appearing in a student-facing page, including one split across markup or echoed onto the portal.                                      |
| accessibility   | WCAG 2.0 A/AA violations, scanned by axe-core in a real browser on every page.                                                                            |
| link-integrity  | Internal links and assets that do not resolve.                                                                                                            |

## What a release looks like on disk

```
/data/releases/2026-08-17T21-04-11-903Z/  
└─ smrttech-3cc3/  
   └─ site/      ← students get this, and only this  
   └─ keys/      ← answer keys; served only to signed-in staff  
   └─ bundles/   ← single-file exports and course ZIPs  
  
/data/current → /data/releases/2026-08-17T21-04-11-903Z/ ← one symlink
```

Old releases are kept on purpose: rolling back is repointing that symlink, which is why the runbook's rollback procedure is three lines long.

### NOW OPEN THE STUDENT SITE

`localhost:8080/c/smrttech-3cc3/lab-01/` — no login, no cookie. That short URL is the canonical one; it redirects to the full path, and it is what you paste into Avenue to Learn. The *Link sheet* page has every one of them with a copy button.

## 5 Being a student

Spend ten minutes here as if you were in the lab. This is the part of the system everything else exists to serve, and most of its interesting behaviour is invisible from the console.

Open `localhost:8080/c/smrtech-3cc3/lab-01/` — ideally in a private window, so you see a genuinely fresh state.

- ☐ **Progression is locked.** Only the first checkpoint is open; later ones show a padlock in the left rail and their controls are disabled. Answer everything in checkpoint one, press *Complete checkpoint*, and watch the next unlock.
- ☐ **Going back un-completes things.** Return to a finished checkpoint and clear a required field. Its confirmation is revoked, and so is every checkpoint after it — completion is derived from the work, never a sticky flag.
- ☐ **The knowledge base never costs you your place.** Click any dotted knowledge link. A panel opens over the lab with the topic in it; close it and nothing has moved. That was the original design goal: reference material must never be a navigation away from your work.
- ☐ **Work survives a reload.** Refresh the page mid-lab. Everything is still there — it is in the browser's local storage.
- ☐ **The progress file is the real record.** Open the *Progress* panel at the page edge, download the file, clear site data, then restore it. This is what a student does when they move between the lab computer and their laptop.
- ☐ **Finish the lab and download the submission package.** The button stays disabled until every checkpoint is confirmed and the identity fields are filled — hover it to see what is still missing.

### Inside the submission ZIP

| Entry                        | Contents                                                                                                          |
|------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>completion.json</code> | Identity, every answer, checkpoint confirmation times, an auto-score, and a SHA-256 integrity hash of the record. |
| <code>evidence/...</code>    | Every file the student attached, named after the evidence slot it belongs to.                                     |

#### PROVE THE PRIVACY CLAIM YOURSELF

Open DevTools → Network, then work through a checkpoint and download the package. You will see the page load and nothing else: no XHR, no beacon, no form post. The marking happens in the page against salted hashes, and the answer key it is checking against is not in the page at all — search the source for the expected value and you will not find it.

## 6 Editing content

The editor is a thin layer over Git. It never becomes a second source of truth: a draft lives outside the repository, and *Apply* turns it into an ordinary commit with your name on it.

**1 Console → Editor → SMRTTECH 3CC3 → *Edit* on Lab 1.**

You get the lab as a list of blocks, grouped by checkpoint, each with move, edit and delete.

**2 Find a `quiz` block, press *Edit*, and reword its `prompt`.**

**3 Press *Save draft*.**

The draft is validated immediately against the real content schemas. A clean draft says so.

**4 Press *Apply to repository*.**

You get a commit hash back. In WebStorm, `Alt + 9` → *Log* shows the commit, authored by the address you signed in with.

**5 Return to the dashboard and *Publish*, then reload the lab page to see your wording live.**

### Now break it on purpose

Edit the same block and delete its `prompt` line entirely, then *Save draft*. The draft still saves — drafts are allowed to be wrong — but a red error appears pinned to that exact block, and *Apply* refuses. This is the same validator the build uses, so the editor and the pipeline can never disagree about what is valid.

### Images

The upload box at the top of the editor requires alt text before it will accept anything, and it identifies files by their actual bytes rather than their name or declared type — a script-bearing SVG renamed `.png` is rejected. On success it stores the image and inserts a figure block.

#### COMMITTS ARE LOCAL UNTIL YOU PUSH

*Apply* commits into the working copy mounted into the container — which is your checkout. Push it yourself when you are happy: `Ctrl + Shift + K`. Nothing in the platform pushes on your behalf.

## 7 Marking submissions

The marker runs entirely in your browser. It re-derives every mark from the student's own recorded answers against the published key — it never trusts the score the student's page calculated.

### 1 Console → Marker.

The keys table fills itself from the live release. If it is empty, you have not published yet.

### 2 Drag the submission ZIP from section 5 onto the drop zone.

It matches the key by lab and content version, marks every item, and shows a total.

### 3 Press Details.

Every item is listed with what the student answered and why it earned what it did.

## What it can mark

| Item type  | How it is judged                                                                                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| choice     | The recorded selection against the key's accepted options.                                                                                                                               |
| value      | Numeric with a tolerance band (and an optional wider band for partial credit), or text with case and whitespace normalised.                                                              |
| formula    | The key's expression evaluated <i>over that student's own measurements</i> — so a wrong resistance reading with an arithmetically correct calculation still earns the calculation marks. |
| evidence   | The file must actually be present in the ZIP, not merely named in the record.                                                                                                            |
| checkpoint | Confirmed, cross-checked against the recorded requirement counts.                                                                                                                        |

## See the formula behaviour for yourself

Unzip the submission, open `completion.json`, change `t3-r-dark` to a different resistance, re-zip, and drop it in again. The expected voltage moves with it: the breakdown will read *“answered 0.455, expected 0.45 from their own inputs — within tolerance”*. That is the difference between marking a lab and marking an answer sheet.

## Advisory flags, never verdicts

The marker raises a flag — and only a flag — when something deserves a human look: a mismatched content version, an integrity hash that does not match, two submissions with identical answers under different names, a checkpoint confirmed with unmet requirements. It never withholds marks on suspicion. The judgement stays yours.

# Exports

| Button             | Gives you                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------|
| Brightspace CSV    | One row per student number, in the format Avenue to Learn's Grades → Import accepts directly. |
| Full breakdown CSV | Every per-item mark and every review flag, for your records or a spreadsheet.                 |
| Feedback files     | A ZIP of one Markdown file per submission, ready to return to students.                       |

## CONFIRM THE NO-EGRESS CLAIM

Open DevTools → Network before you drop a submission. The only requests you will ever see are the `/keys...` fetches at load. The page's Content-Security-Policy forbids every other destination, the server has nowhere to receive submissions, and a test in the suite fails the build if anyone adds a network call to this app.

---

## 8 Single-file exports

Every publish also produces each lab as one self-contained HTML file — stylesheet, fonts, scripts and images folded in — for when a lab has to live inside the LMS rather than behind a link.

1 Console → Exports → download `smrttech-3cc3-lab-01-single.html` .

2 Open it straight from your downloads folder, with no server involved.

It is a complete lab: quizzes mark, checkpoints gate, the progress file downloads. Disconnect your network first if you want to be certain.

### TEACH THE STORAGE CAVEAT, OR FIELD THE SUPPORT TICKETS

Browsers separate storage by origin. Work done in a single-file copy stays in that copy — it will not appear on the hosted site, and vice versa. The progress file is the bridge between them. If a course uses both forms, one sentence in the Avenue module text prevents nearly every confused email: *“your work saves in whichever copy you use; use the progress file to move it.”*

The files are large — roughly 20–25 MB, dominated by embedded images — which is fine for an LMS file topic and poor for email. Prefer the canonical link whenever the choice is yours.

## 9 Running the tests

Eighty-six tests across four suites. They run on your machine rather than in the container, so you need Node once — and then one command.

```
# once, in the repository root
npm run setup

# thereafter
npm run test:all
```

`setup` installs the three packages that have dependencies and fetches the Chromium build the accessibility and export tests drive. In WebStorm you can skip the terminal entirely: open the root `package.json` and use the run arrows in the gutter beside each script, or the `npm` tool window.

*What each suite is guarding*

| Suite                               | Tests | Protects                                                                                                                                                    |
|-------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>npm test</code><br>(renderer) | 29    | The hashing scheme, tolerance edges, key generation, the answer-leak gate, and single-file export parity against the hosted page.                           |
| <code>npm run test:platform</code>  | 10    | Magic links, sessions, roles, CSRF, rate limiting, path-traversal defences, the absence of student routes, and an accessibility scan of every console page. |
| <code>npm run test:marker</code>    | 41    | Every marking rule, the formula evaluator's safety, ZIP reading, CSV formatting, and the no-egress guarantee.                                               |
| <code>npm run test:admin</code>     | 6     | Block skeletons stay schema-valid, the draft→validate→apply flow, course access, and upload hardening.                                                      |

### Running the gates directly

To exercise the full build without the console, run `npm run gates` (the pilot course in strict mode) or `npm run build:all` (both courses, plus single-file exports). This is what CI runs on every push, and it is the fastest way to check a content change before publishing.

#### EXPECT THE SQLITE NOTICE

On Node 22 the platform suite prints *"SQLite is an experimental feature"*. That is Node talking about its own built-in database module, not a problem with the platform, and it disappears on newer Node. The `npm test` scripts pick the right flag for whichever version you have.

## 10 Proving it works

This is the acceptance script the platform was built against, written out so you can walk it by hand. Roughly twenty minutes. If all eight boxes tick, every claim in this guide holds on your machine.

- ☐ **1 • The stack comes up seeded.** `docker compose up -d`, and the log ends in *listening*. Mailpit holds two sign-in emails.
- ☐ **2 • An invitation reaches a new person.** Sign in as admin. Invites → invite `tester@demo` as an instructor for SMRTTECH 3CC3. The link appears in Mailpit; open it in a private window and you are signed in as them, with a narrower console.
- ☐ **3 • An edit reaches students only through the gates.** As that instructor, reword a Lab 1 quiz prompt in the editor → Save draft → Apply. Publish as admin. The report shows every gate passing, and `localhost:8080/c/smrtech-3cc3/lab-01/` shows the new wording.
- ☐ **4 • A student completes the lab with no account.** In a fresh private window, work through Lab 1. Put one measurement deliberately near the edge of its tolerance. Fill the identity fields, attach a file to each evidence slot, download the submission package.
- ☐ **5 • The marker marks it.** Back as the instructor, Marker → drop that ZIP. The key matches automatically, the formula items are judged against that student's own measurements, and the Brightspace CSV downloads with one row per student number.
- ☐ **6 • Answer keys stay inside their course.** As an instructor invited to only one course, request another course's key with a traversal: `/keys/smrtech-3cc3/..%2f..%2f..%2fdemo-101%2fkeys%2fdemo-101%2flab-01.key.json`. It must refuse — `400`, and no key contents.
- ☐ **7 • A single-file export works with no server.** Exports → download the Lab 1 single file → open from disk, offline. Quizzes still mark.
- ☐ **8 • The answer-leak gate really blocks a leak.** Add the text `4.68` to the footer in `renderer/src/pages/Page.js`, then run `node renderer/src/cli.js build content/courses/demo-101 --out /tmp/leak --skip-ally`. The build must fail on `answer-leak`. Undo the edit and it passes again.

### WHAT THIS ESTABLISHES

Content only reaches students through the gates; students need no account and leave no data on the server; marking is reproducible from what the student actually recorded; and answers cannot leak, either into a page or across a course boundary. Those are the four claims the pilot rests on.

## 11 When something breaks

Symptoms you are most likely to hit first, and what each one actually means.

| Symptom                                                      | Cause                                                                        | Fix                                                                                                                                                     |
|--------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exec /entrypoint.sh: no such file or directory</code>  | Windows CRLF line endings in the shell script.                               | Rebuild the image (the fix is baked in):<br><code>docker compose build --no-cache platform</code> . For the checkout itself, see section 1.             |
| A container exits the moment it starts                       | Port 8080 or 8025 already taken.                                             | Free the port, or change the left-hand number in <code>docker-compose.yml</code> (e.g. <code>8090:8080</code> ) and set <code>BASE_URL</code> to match. |
| Platform starts, but says it can find no courses             | The repository is not shared with Docker, so the mount is empty.             | Docker Desktop → Settings → Resources → File Sharing → add the parent folder → restart Docker Desktop.                                                  |
| Student site says <i>Nothing published yet</i>               | Accurate — no release exists.                                                | Sign in as admin and press Publish.                                                                                                                     |
| No sign-in email                                             | Mailpit was still booting, or the address has no account.                    | Check <code>localhost:8025</code> first. Unknown addresses are silently ignored, on purpose — the platform will not confirm who has an account.         |
| Publish fails on <b>accessibility</b> with a missing browser | The image drifted from the Playwright version in <code>package.json</code> . | <code>docker compose build --no-cache platform</code> .                                                                                                 |
| Publish fails on <b>answer-leak</b>                          | Not a bug — a marked answer is printed in a student page.                    | The report names the file and the exact value. Fix the content. Never relax the gate.                                                                   |
| Publish fails on <b>schema</b> right after an edit           | Content edited outside the editor, bypassing validation.                     | Run <code>npm run validate</code> to see the same errors locally, with paths.                                                                           |
| <code>ERR_UNKNOWN_BUILTIN_MODULE: node:sqlite</code>         | Tests started with a bare <code>node --test</code> on Node 22.               | Use <code>npm run test:platform</code> , which selects the right flag for your Node.                                                                    |
| Out of disk                                                  | Releases accumulate by design (they are the rollback history).               | <code>docker compose exec platform sh -c 'ls -dt /data/releases/*   tail -n +6   xargs rm -rf'</code>                                                   |

### Reading logs in WebStorm

Services (`Alt + 8`) → the container → *Log* tab for output, *Terminal* tab for a shell inside it. To inspect a release from the inside:

```
ls -la /data  
ls /data/current/smrtech-3cc3/site
```

## 12 Reference

### URLs

| URL                                                       | What                                                               |
|-----------------------------------------------------------|--------------------------------------------------------------------|
| <code>localhost:8080/c/&lt;course&gt;/&lt;lab&gt;/</code> | Canonical student lab link — this is what goes in Avenue to Learn. |
| <code>localhost:8080/login</code>                         | Faculty sign-in.                                                   |
| <code>localhost:8080/admin</code>                         | Console dashboard, publishing, publish history.                    |
| <code>localhost:8080/admin/links</code>                   | Link sheet with copy buttons and suggested A2L titles.             |
| <code>localhost:8080/admin/exports</code>                 | Single-file labs and course ZIPs.                                  |
| <code>localhost:8080/admin/editor</code>                  | Content editor.                                                    |
| <code>localhost:8080/marker/</code>                       | Instructor auto-marker.                                            |
| <code>localhost:8080/keys/&lt;course&gt;</code>           | Answer keys as JSON — role-gated.                                  |
| <code>localhost:8080/healthz</code>                       | Liveness plus the current release path.                            |
| <code>localhost:8025</code>                               | Mailpit.                                                           |

### Environment variables

| Variable                                        | Default                                  | Purpose                                                                                                                                                                            |
|-------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>BASE_URL</code>                           | <code>http://localhost:8080</code>       | The origin written into magic links and the link sheet. Must be right, or emailed links point nowhere.                                                                             |
| <code>COOKIE_SECRET</code>                      | dev value                                | Signs sessions and CSRF tokens. <b>The service refuses to start with the default on any non-localhost <code>BASE_URL</code>.</b> Generate with <code>openssl rand -hex 32</code> . |
| <code>SEED_ADMIN</code>                         | <code>admin@demo</code>                  | Administrator seeded on start.                                                                                                                                                     |
| <code>SEED_INSTRUCTOR</code>                    | <code>instructor@demo</code>             | Instructor seeded on start.                                                                                                                                                        |
| <code>SMTP_HOST</code> / <code>SMTP_PORT</code> | <code>mailpit</code> / <code>1025</code> | Where sign-in mail goes. Point at a real relay in production.                                                                                                                      |
| <code>PUBLISH_GIT_PULL</code>                   | off                                      | Set to <code>1</code> to <code>git pull --ff-only</code> before each publish.                                                                                                      |
| <code>PUBLISH_SKIP_A11Y</code>                  | off                                      | Skips the accessibility gate. For environments with no browser only — never for a real publish.                                                                                    |

## Where things live

| Path                          | Contents                                                              |
|-------------------------------|-----------------------------------------------------------------------|
| <code>content/courses/</code> | The courses. This is the durable asset; everything else is machinery. |
| <code>schema/v1/</code>       | JSON schemas for labs, blocks, knowledge, progress, answer keys.      |
| <code>design-system/</code>   | The one stylesheet and the browser runtime, in layers.                |
| <code>renderer/</code>        | Content → static site, plus every quality gate.                       |
| <code>apps/platform/</code>   | The faculty service: auth, publishing, keys, exports.                 |
| <code>apps/admin/</code>      | The content editor mounted by the platform.                           |
| <code>apps/marker/</code>     | The client-side auto-marker.                                          |
| <code>deploy/</code>          | Compose file, image, reverse proxy, seed script.                      |
| <code>integrations/</code>    | Phase B seams — interfaces only, nothing wired up.                    |
| <code>docs/</code>            | runbook · architecture · marking · deployment · decisions/adr-001-cms |

## The five invariants, and what enforces each

| Invariant                                            | Enforcement                                                                                                                          |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| No student data on the server                        | No student route exists; a test asserts every plausible submission endpoint returns 404/405.                                         |
| No plaintext answers in public artifacts             | The answer-leak gate, on every build, structurally and by text — including answers split across markup.                              |
| Git is the source of truth; builds are deterministic | The editor commits rather than storing; the build timestamp comes from the content commit, verified by byte-identical rebuilds.      |
| All styling through the design system                | The inline-style and class-allowlist gates; the console uses the same stylesheet and passes the same accessibility scan.             |
| One clearly marked authentication seam               | A single <code>OIDC SEAM</code> block in <code>apps/platform/src/auth.js</code> ; roles and course membership stay local either way. |

## 13 Honest status

What you are running is a complete Phase A prototype, verified end to end — not a finished production service. The distinction matters most in these places.

### Solid **VERIFIED**

The rendering pipeline and its gates; the marking model and the marker; content editing through Git; publishing with atomic rollback; the student experience including gating, progress files and submission packages. All covered by tests and by the section 10 walkthrough.

### Stubbed **BY DESIGN**

Authentication is email magic links, not MacID — deliberate, and confined to one seam. Grade return is a CSV you import by hand. Both are Phase B swaps, and their contracts already exist under [integrations/](#).

### Rough edges

Publishing rebuilds every course, not just the changed one. The editor edits blocks as JSON rather than generated forms (the reasoning is in ADR-001). Rollback is a documented command, not a button.

### Decide before the pilot

Where this is hosted and who administers it; whether single-file exports are used at all, given the storage caveat; who holds the admin account; and how student evidence files are retained once they reach Avenue to Learn.

## If you do only three things before students see it

1. Set a real `COOKIE_SECRET` and a real `BASE_URL`, and put the service behind HTTPS.
2. Point `SMTP_HOST` at a mail relay that can actually reach McMaster addresses, and invite yourself to prove it.
3. Walk section 10 on the machine that will host it — not on your laptop.

### WHERE TO GO DEEPER

[docs/runbook.md](#) for day-to-day operations and rollback · [docs/architecture.md](#) for how the layers fit · [docs/marking.md](#) for the hashing scheme and its honest security claims · [docs/deployment.md](#) for hosting options · [docs/decisions/adr-001-cms.md](#) for why the CMS is what it is.